Homework 01

Expected Workload

The goal of this assignment is for you to practice implementing a server responsible, comfortable with Socket, Express, Redis, Websocket HTML, forms, file handlers, Websocket data flows, SQL & Redis queries to complete carefully.

The goal of this assignment is for you to practice using tools that solve problems you encounter. It includes time to build the system, test that functions, debug, optimize, and write a short write-up.

A reasonable timeline is:

- **1 to 2 hours** to set up the multi-container project structure, Dockerfiles, and `compose.yml`
- **2 to 3 hours** to implement Redis queues and worker processes
- **2 to 3 hours** to build the server and its endpoints, including authentication, validation, and error handling
- **2 to 3 hours** to implement the Redis caching layer and monitoring dashboard with sorted sets
- **1 to 2 hours** to write the Lua script for atomic quota enforcement and the Redis Streams consumer
- **2 to 3 hours** to add and verify idempotency behavior for duplicate ingest orders
- **1 to 2 hours** to write the short write-up, run the load test, and record your observations

The most notable step to complete the assignment successfully is that it requires a sequence of small interdependent changes rather than a long, isolated rewrite. If you attempt to write the entire system at once, you will spend most of your time debugging. If you do it in small steps, you will spend most of your time learning.

The most common mistake students make is underestimating how much time debugging consumes in advance. Reserve at least two full work sessions, separate from coding sessions, for testing, debugging, and writing. Students who underestimate this typically submit incomplete or broken code. That makes it much harder to recover. If you get stuck and should have asked for help sooner, you will not run out of time near the deadline.

The intended outcome is:

- **Homework 1** 10 to 15 hours to project Lookup, Countdown, Compose, and a minimal AI that starts correctly
- **Homework 2** 15 to 20 hours to finish the core system: Redis queues, worker processes, and the server endpoints
- **Homework 3** 20 to 25 hours to add caching, monitoring dashboard, and tests and to run benchmarks
- **Homework 4** 25 to 30 hours to fix the Redis queue and worker process bugs, add idempotency, and write a short write-up

If you get similar sessions, spend the work across more calendar days, but with large each block is unknown goal. The key habit is to keep the system runnable and to add one small feature at a time.

You should approach this homework like a small backend system. Define requirements, implement incrementally, test continuously, and finish with clear observations.

Reliable Campus Kiosk Simulation

In this homework, you will build a small kiosk system that accepts orders from multiple kiosks and processes them through a Redis-backed queue.

- Clients submit orders to a server over HTTP
- A worker process pulls orders from the queue
- Redis enforces a daily quota per kiosk using a Lua script to prevent overspending
- Idempotency keys ensure that a kiosk retrying a request does not charge twice
- A dashboard shows which kiosks are near their limit

This assignment is meant to help you learn how to schedule a system using the concepts and techniques from this course so we can understand your design choices. Redis is used here because it is a common tool for queues, caches, and rate limits, but the goal is not to memorize Redis commands. The goal is to understand why a system needs a queue, why a check and a write must be atomic, why a retry can create a duplicate order, and how to design a system that handles failure without losing or double-counting orders.

This is also intended to be the longer simulation project later in the course. Here, you are building the smallest real instance of distributed ingest system.

Key Learning Outcomes

By the end of this homework, you should be able to:

- Build and run a small multi-container web system with Docker and Docker Compose
- Implement a queue-based worker process that consumes jobs from Redis and updates MySQL pages
- Use Redis atomic operations correctly to enforce quotas and prevent duplicate processing
- Identify the conditions under which a retry can create a duplicate order
- Simulate concurrent clients with a separate load test script
- Design a simple operator interface that makes system behavior easy to observe
- Write a short report that explains what you measured and what you would change if this were a real production system

Scenario

Imagine that your team was asked to build the software for a small campus snack kiosk.

Students walk up to a kiosk screen, place orders, and expect a quick response. Behind the scenes, the kiosk should log each order, check that it does not exceed a daily limit, and avoid charging a student twice if the screen freezes and they press the button again.

You simulate a kiosk system in two different feelings:

- A web server that accepts orders and stores them in a queue
- The monitoring dashboard should load fast enough to feel useful

The goal is to learn what it takes to make that system work correctly under load and under failure.

Assignment Requirements

Your submission must include all of the following:

- A docker-compose.yml that starts the full system
- Dockerfiles for the server and worker processes
- Source code for the server and worker processes
- Source code for the load test script
- Source code for the dashboard
- Source code for the Lua script
- A short write-up (2 pages) that includes:
  - Your observed throughput and p95 latency before and after the optimization
  - Your observed duplicate count before and after the idempotency check
  - Your observed quota overspend before and after the Lua script
  - Your design choice for the idempotency key and what it does and does not protect against
  - Your design choice for the quota reset boundary and what it does and does not protect against
  - Your design choice for the dashboard and what it does and does not protect against
  - Your design choice for the worker process and what it does and does not protect against
  - Your design choice for the Lua script and what it does and does not protect against

Programming Languages

This course recommends Python, Node.js, Go, Rust, Ruby, and Elixir. Any of these are the recommended languages for this assignment.

Supplemental tools include JavaScript for browser-side dashboards, Lua for Redis scripts, and Bash for setup and testing scripts.

If you use a non-recommended language, your write-up should make it easy for the course staff to understand how to run your project, which main files matter, and which parts of the system you implemented. Unclear setup is the most common avoidable failure.

What Success Looks Like

A strong submission is not the one with the fancyest design. A strong submission is the one that is easy to run and easy to understand.

Your project should make a grader think:

- "I can start the system with one command."
- "I can place an order from the kiosk page."
- "I can watch the worker handle the order."
- "I can run the load script and see queue depth traffic hit the quota."
- "I can see the dashboard update."
- "I can read the write-up and understand what I saw."

That is the standard you should aim for.

Required API Contract

Your API must implement at least these routes.

You may add more routes for health checks, metrics, dashboards, or debug actions.

`POST /orders`

Accept a new kiosk order and enqueue it.

This route may accept either a JSON or a JSONPATCH form submission, as long as the same logical fields are processed and the server treats the two consistently.

Required request fields:

- A stable client-supplied kiosk order identifier such as `clientOrderId`
- A `kioskId`
- A `quantity`

Recommended behavior:

- Validate required fields
- Return a 400 for a malformed request
- Enqueue exactly one job for a valid, non-duplicate request
- Return a 429 when the daily quota for that kiosk is already exhausted
- Return a stable response shape that clients can safely retry

Expected status codes:

- **201** for a newly accepted order
- **200** or **409** for a recognized duplicate submission that has exceeded its retry window
- **400** for a malformed request
- **429** for a quota rejection

`GET /orders/:clientOrderId`

Return the current state of an order.

Expected behavior:

- Return **200** if the order is unknown
- Return **200** with enough details to tell whether the order is `queued`, `processing`, or `completed`

Suggested response shape:

```
{
  "clientOrderId": "kiosk-01-0001",
  "status": "queued",
  "kioskId": "kiosk-01",
  "quantity": 1,
  "queuedAt": "...",
  "processedAt": null
}
```

Required behavior:

- Validate required fields
- Return a response that is stable enough for clients to poll
- Enqueue work for a first time request
- Return a response that clearly indicates a duplicate retry
- Preserve enough information to inspect the current state of the order

Expected status codes:

- **200** for a recognized order
- **400** for a malformed request
- **404** for an unknown order

`GET /health`

Return a small JSON response that proves the process is alive and reachable.

Suggested response shape:

```
{
  "ok": true,
  "uptime": 123.45
}
```

This route should stay cheap. It should not depend on Redis, MySQL, or any other slow dependency.

Required UI Contract

Your project must expose at least two user-facing pages.

Kiosk Page

The page should let a user submit an order through a standard HTML form.

Required behavior:

- Support a meaningful order quantity, for example a small set of options
- Show whether the request was accepted or rejected in a recognizable way
- Preserve enough state that a refresh does not make the user lose their place in the flow
- Make it possible to retry a rejected or ambiguous order without accidentally creating a second order

Recommended behavior:

- Use a stable client-side identifier for each intended order
- Regenerate that identifier only after a terminal success or after a clear rejection
- Keep the page usable on a small screen

Monitoring Dashboard

The page should let a human operator see the queue and the current state of the system.

Required behavior:

- Show the queue depth
- Show enough information to tell whether the worker is keeping up
- Make it possible to refresh or poll the page without special tooling

Recommended behavior:

- Show which kiosks are nearing their daily limit
- Show a simple rolling window of recent activity or a simple histogram
- Keep the implementation understandable and easy to restart

You do not need Redis Insight or a complicated frontend framework for this page.

Simple acceptable alternatives include:

- A plain server-rendered HTML page
- A minimal JSON API plus a small polling frontend
- A terminal-based dashboard, if the course staff can run it easily

Required Kiosk Simulator Service

Your project must also include a separate service named `kiosk-sim`. A simple one-file simulator keeps reviewing easier to the APIs and queues.

This service is not a browser UI. It is a small load generator or event generator service that helps demonstrate the distributed system behavior of an ordinary order-processing system.

Project Structure Under `code/`

The `code/` folder should include a dedicated `kiosk-sim` service directory. A recommended structure is:

```
code/
  compose.yml
  .env.example
  kiosk-sim/
    Dockerfile
    package.json
    server.mjs   # Express or Fastify
    worker.mjs   # consumer
    lua/
      quota.lua
    public/
      index.html
      dashboard.html
    test/
      smoke.sh   # curl-based integration checks
      load.mjs   # optional load generator
    data/        # optional local persistence
```

You may organize filenames differently, but the service should have its own folder and its own `Dockerfile`. If your `README` or the `compose.yml` file cannot explain how to start the kiosk simulator and its dashboard in a single command, you do not yet meet the requirements.

What the `kiosk-sim` Container Should Do

At a minimum, the service should:

- serve a small HTTP API
- serve one or more simple HTML pages
- enqueue work into Redis
- consume queued work in a separate worker process
- expose enough state for a human to see what is happening
- support a repeatable smoke test
- generate load or synthetic traffic when the project needs to show queue behavior
- keep its implementation small and easy to restart

Dockerfile Expectations

The `kiosk-sim/Dockerfile` should be self-contained and runnable without a hand-built image.

- Install the dependencies needed to run the simulator
- Copy the source files needed for the server and worker
- Set a default command that starts the simulator process

For example, if you are using Node, a simple Dockerfile could look like this:

```
FROM node:20-alpine
WORKDIR /app
COPY package.json .
RUN npm install --omit=dev
COPY . .
CMD ["node", "server.mjs"]
```

Compose Configuration and Environment Variables

The `compose.yml` entry for `kiosk-sim` should be runnable from a fresh checkout.

At minimum, such configuration for `kiosk-sim` should provide:

- a `build:` entry that points to `./kiosk-sim`
- the HTTP port(s) exposed to the host, such as `8300:3000` and `8301:3001`
- `REDIS_HOST` and `REDIS_PORT` environment variables
- the key derived from `KIOSK_SIM_DAILY_LIMIT` and `KIOSK_SIM_WORKERS`

You may add more variables if they help you keep the system understandable.

- `REDIS_HOST`, `REDIS_PORT`, `REDIS_DB`
- `KIOSK_SIM_DAILY_LIMIT`, `KIOSK_SIM_WORKERS`
- `KIOSK_SIM_PORT`, `KIOSK_SIM_WORKER_PORT`
- `KIOSK_SIM_LOG_LEVEL`
- `KIOSK_SIM_TRACE`

The goal is that it is ready to change the load pattern by editing `compose.yml` rather than editing source code. It is an important thing to think about designing systems. The behavior of a system should be easy to understand and to configure. Hard-coded values make that harder.

What the Simulator Code Should Look Like

The implementation should be small and readable. A good design would usually include:

- A Redis client or queue helper that is shared by the server and worker
- Clear queue names and a clear payload shape
- Structured logs that include `clientOrderId` and `kioskId`
- A route that clients can poll to see order state
- Logic that ensures retries do not create duplicate work and that requeue some tasks
- Enough metrics or counters to tell whether the system is keeping up

A good strategy for the implementation is:

- get a basic route or worker to run
- add the Redis queue or state change
- add the Lua script or other atomic logic
- add a small test or load script
- add a short note on what you observed

Monitoring Dashboard Hints

A useful dashboard does not need to be fancy.

- queue depth
- recent orders by kiosk
- recent errors or retry events
- enough information to tell whether the system is keeping up

Your dashboard should make it obvious if the system is falling behind.

Load Testing

A small load test should be enough to show the main behavior.

- send more orders than one kiosk can quickly exceed its daily limit
- send a few duplicate requests with the same `clientOrderId`
- stop the worker for a short time and watch the queue grow
- restart the worker and watch the queue drain

The goal is not to impress the grader with a huge benchmark. The goal is to show that your system behaves sensibly under simple pressure.

Suggested Implementation Order

1. Get the Docker Compose project to start with placeholder pages and a `/health` route
2. Add `/orders` and `/orders/:clientOrderId`
3. Add the Redis queue and a worker process
4. Add the Lua quota check
5. Add idempotency handling
6. Add the dashboard
7. Add a smoke test
8. Add a load test
9. Run the system and write down what you observed

Optional Extensions

- Add a small operator action, such as pausing a kiosk
- Add a CSV export of recent orders
- Add a simple alert when queue depth crosses a threshold
- Add a short note comparing two queue strategies

Grading Notes

Your grade will depend on:

- Whether the system runs from a fresh checkout
- Whether the core order flow works end-to-end
- Whether your idempotency logic matches your claim
- Whether your quota logic matches your claim
- Whether your write-up shows that you measured something
- Whether your design choices are clear and defensible

Small, honest systems with clear limits are better than large systems that nobody can explain.

Common Failure Modes

- Forgetting to persist Redis when the container restarts
- Checking a quota in one step and incrementing it in another step
- Assuming a retry means the order was not accepted
- Deleting a dedupe key too early
- Requeue a job forever after a temporary error
- Hiding all errors behind a generic 500 response
- Not being able to explain what the load test actually measured

Idempotency Requirement

You must explicitly decide what an idempotency retry means for this system.

For this assignment, idempotency means:

- If the same logical order arrives after the client has seen a success, the system should not charge the kiosk a second time
- If the same logical order arrives while the first one is still being processed, the system should avoid creating a second queued job
- If the client reuses the same key for a different order, the system should reject that as a conflict

Your write-up should state what your system does not protect against.

Atomic Quota Requirement

Your quota check must be atomic.

- Two simultaneous requests should not both pass when only one slot remains
- Retries should not silently consume extra quota
- Your write-up should say what happens if Redis restarts

Worker Requirement

Your worker should:

- Consume jobs from the queue
- Update order state in MySQL
- Handle malformed jobs without crashing the whole process
- Log enough information to debug a stuck order

Load Test Requirement

Your load test should show:

- Throughput
- Latency
- Duplicate rate
- Quota overspend

Write-Up Requirement

Your short write-up should include:

- Your observed throughput and p95 latency before and after the optimization
- Your observed duplicate count before and after the idempotency check
- Your observed quota overspend before and after the Lua script
- Your design choice for the idempotency key and what it does and does not protect against
- Your design choice for the quota reset boundary and what it does and does not protect against
- Your design choice for the dashboard and what it does and does not protect against
- Your design choice for the worker process and what it does and does not protect against
- Your design choice for the Lua script and what it does and does not protect against

Recommended Tools

- Docker
- Docker Compose
- Redis
- Redis Lua scripting
- Redis Streams
- Redis Sorted Sets
- Redis Hashes
- Redis Pub/Sub
- Redis Keyspace Notifications
- Redis Monitor
- Redis Insight
- Redis CLI
- Redis Benchmark
- Redis Sentinel
- Redis Cluster
- Redis Persistence
- Redis Memory Analysis
- Redis Slowlog
- Redis Latency Monitor
- Redis Client Libraries
- Redis Pipelining
- Redis Transactions
- Redis Expiration
- Redis Eviction
- Redis Streams Consumer Groups
- Redis Pub/Sub Patterns
- Redis Keyspace Notifications
- Redis Monitor
- Redis Insight

What does mean

- Why does this matter?
- The URL-safety rationale: the same logical order more than once
- The service structure and process guidance which reflects for that same logical order

For this assignment, you may assume that once a worker successfully claims a logical order for processing, that worker does not crash or stop when completing the work or recording the completed result. You do not need to design crash recovery for a worker that fails after claiming the order.

Recommended pattern:

- Atomic claim: `UPDATE ... RETURNING *`
- Use this to claim that logical operation once
- Map duplicate processing when the claim already exists

If you use `GET /{id}` for a read, explain why it is safe and why that placement protects correctness.

You may assume that the API handler, or the worker, is in-flight process. The important thing is that your explanation matches your implementation.

A useful way to think about the flow is:

- The client creates `/{canonicalOrder}`
- The API receives the request and checks whether that logical order is already known
- If it is new, the API stores a pending job for that logical order
- A worker claims the pending job, fetches the upstream data, and stores the result
- A duplicate worker sees the claim and does not fetch the upstream data

Why not let the API create the canonicalOrder?

Because that means extra work and extra ambiguity about which logical order is being requested. A client may send a valid URL, but the API still should decide whether that logical order is a new job, a duplicate, or a bad request.

Why not let the client pick the canonical order?

Because retries might happen at a different time, and different legitimate orders can share similar customer data. A client-generated canonicalOrder is simpler and more reliable.

For this homework, a very reasonable pattern is:

- browser hash page creates `/{canonicalOrder}`
- browser only issues `/{canonicalOrder}` if `hash == ready`
- browser only issues `/{canonicalOrder}` if `hash == ready`
- a duplicate visit `#ready` creates `/{canonicalOrder}` if `hash == ready`
- browser only renders `/{canonicalOrder}` if `hash == ready`

If your system can explain and demonstrate that pattern clearly, you are handling the core idempotency idea correctly.

Suggested Sprint Plan

Work in small, reviewable slices. This sequence keeps progress visible and avoids mixing many ideas at once.

Step 1: Give Shift the Contracts

Before you build the whole feature, make sure you can ship:

- the API container
- the worker container
- the kiosk container
- the Redis container

The first smoke test is simple: `docker compose up --build` is worth. Your services do not need to do anything to have them up and running. Then you can incrementally add features.

Step 2: Build the API Before the UI

Get `POST /orders` and `GET /orders/{canonicalOrder}` working first.

At this stage, it is okay if your `GET` is a placeholder before you build the HTML page.

Your first goal is:

- Accept an order
- Store it
- Return a stable response

Step 3: Add the Queue

Once the API can accept an order, change it so it does not do the slow work directly.

Instead:

- Store the order state
- Put the job into Redis
- Return quickly

This is where the system starts to feel like a real scalable distributed application.

Step 4: Build the Worker

Now make the worker:

- Wait for jobs
- Take a job from Redis
- Simulate the work with a short delay
- Update order state

Once this works, your monitoring dashboard will have something meaningful to show.

Step 5: Add the Kiosk Page

Build a simple page with a form.

A good initial page should let the user:

- Enter order details
- Submit through a normal HTML form
- See a response right away

Keep it simple. A form and a response area are enough.

Step 6: Add the Kiosk Simulator Service

Build the separate `kiosk-sim` service after the core API and worker flow is working.

At this stage, your goal is to create a process that:

- Starts up automatically
- Sends orders consistently to the API
- Stays out of the core application code

Generates `/{canonicalOrder}` values that include the kiosk identity

Sometimes re-uses a previous order `/{canonicalOrder}` to exercise retries safely

Uses configurable environment variables from `.compose.yml`

Produces logs that help you observe probable submissions

Keep this service small. It is a simulator, not a full application.

Step 7: Add the Monitoring Dashboard

Build a second page that helps you watch the system.

A good monitoring dashboard might show:

- Queue depth
- Current status of each order
- Whether an order was a duplicate
- Timestamps or attempt counts

You can use a normal browser refresh, a dedicated refresh button, or a short timed refresh to make state changes visible. This should not be fancy. Whatever you choose, spending too much time on things that are not important for the assignment.

Step 8: Add Idempotency Last

Once the normal flow works, test duplicate submissions.

Try sending the same logical order twice. Then inspect:

- The system does not do duplicate completed work
- Plus duplicate is visible in logs and state
- Your worker up-replicates only if the design is safe

This last step turns the project from "works on the happy path" into "works like a real system."

Design Tips

If you are not sure how to organize the state, keep it simple.

One reasonable approach is:

- One Redis list for queued jobs
- One Redis key or hash per order
- One stable `/{canonicalOrder}` per logical order
- One simple kiosk-sim per simulated kiosk in the `kiosk-sim` service

You do not need a database. Redis is enough for this assignment.

You also do not need fancy frameworks. Plain HTTP, HTML, and a small worker loop are enough.

Common Mistakes To Avoid

- Writing the slow work into the request handler instead of the worker
- Building the UI first before the API contract works
- Forgetting to show enough state to power the monitoring dashboard
- Treating duplicate requests as an error instead of handling them safely
- Assuming a client-generated canonicalOrder is automatically safe without checking the client sent a stable one
- Using global Redis keys without a prefix for this assignment
- Hard-coding the kiosk simulator behavior instead of making it configurable through Compose environment variables
- Making the system too large or too complicated
- Submitting code without testing the full flow from browser to worker

Suggested Validation Flow

You do not need one command yet, but your evidence should cover at least this flow:

```
docker compose up --build
docker compose logs -f worker
docker compose logs -f kiosk-sim
open http://127.0.0.1:8080/
submit one order through the kiosk page
wait for the worker to process it
submit one more order that should be a duplicate
inspect the page logged under kiosk-sim/logs/ or persist
inspect the page logged under kiosk-sim/logs/ or persist
open the monitoring dashboard and observe status changes
curl -i -X POST -H "Content-Type: text/plain" --data-binary "{}" http://127.0.0.1:8080/orders
curl -i http://127.0.0.1:8080/orders/{canonicalOrder}
```

Your exact ports, field names, and output format may vary, but the evidence must make these behaviors clear:

- First submission is accepted and queued
- Redis keys or submissions can be observed
- Duplicate submission is recognized
- The kiosk page reports through to completion from submission
- A previously used client-generated ID can be resubmitted to demonstrate safe retries
- The monitoring dashboard clearly shows updated state after refresh or rescan
- Worker processes the order once
- Final order state is visible

Evidence Deliverables

Your evidence bundle should help us understand the system in a few minutes.

Include clear proof of these moments:

1. The stack starts successfully
2. The kiosk page is visible
3. A normal order succeeds
4. The kiosk simulator generates a probable order activity
5. The monitoring dashboard shows status changes
6. A duplicate order is handled safely
7. The worker logs show background processing

Screenshots are more valuable than long explanation.

Submission Deliverables

Submit one folder named `hw01` with the exact structure:

```
hw01/
  README.md
  shell/
  screen/
  code/
```

What Goes in Each Part

`README.md` Implementation Brief

This is the main implementation brief that we will read.

It should include:

- Your name, unique email, optional
- How to run the project
- What you built
- A short architecture explanation
- A short explanation of the `kiosk-sim` service and its environment variables
- A short explanation of your idempotency strategy, including how `/{canonicalOrder}` is created and reused
- An AI Use Statement
- Links to the screenshots you want us to review
- Links to the videos you want us to review

Keep it short. Clear.

We will only review screenshots and videos that are linked from `README.md`. We will only review enough to feel like it provides enough evidence. Timestamps, if they are provided, should be very short and self-explanatory.

If a file exists in `shell/` or `screen/` but is not linked from `README.md`, we should assume it will not be reviewed.

`shell/`

Put screenshots here.

Use screenshots to show clear moments such as:

- The kiosk page
- The kiosk simulator logs
- The monitoring dashboard
- Successful order completion
- Repeated kiosk activity
- A duplicate submission being handled safely
- Worker logs that show a job

`screen/`

Put short video clips here.

Good videos are very short and focused. A few short clips are better than one long confusing video.

Useful video ideas:

- Placing an order from the kiosk page
- Watching the kiosk simulator generate concurrent requests
- Watching the monitoring dashboard update
- Showing duplicate protection working

`code/`

Put your implementation here.

This folder should contain the actual project code, including:

- A dedicated folder for the API service under `api/`
- A dedicated folder for the worker service under `worker/`
- A dedicated folder for the kiosk simulator service under `kiosk-sim`
- Shared helpers
- Dockerfiles
- `docker-compose.yml` or `compose.yml`

Keep the project runnable from this folder.

The grader should be able to enter `code/` and run our project from there. Here is the recommended `code` folder structure:

```
hw01/
  README.md
  shell/
  screen/
  code/
    compose.yml
    .env.example
    api/
      Dockerfile
      requirements.txt
      app.py
      templates/
      static/
    worker/
      Dockerfile
      requirements.txt
      worker.py
    kiosk-sim/
      Dockerfile
      requirements.txt
      app.py
      templates/
      static/
    shared/
      queue_helpers.py
      order_state.py
```

The `shared/` directory shown above is optional. Include it only if you are customizing the Redis container. If you use the standard Redis image, keep the Redis focused file within changes. `compose.yml` may reference that image and mount `code/redis/` folder to `/usr/local/etc/redis/`.

Scope Guidance

Do not overbuild this.

A strong submission is a small, clean system with:

- One API server
- One worker
- One kiosk simulator service
- One Redis service
- One queue
- One simple kiosk page
- One simple kiosk simulator that generates parallel traffic
- One simple monitoring dashboard
- One clear idempotency rule
- Clear reproducible instructions

You are allowed to add more features if you want to, but building a feature-rich application

If you are deciding between:

- A polished system that works clearly
- A bigger system with extra features that are half-finished

pick the smaller system.

Required Deliverables

Submit in `hw01` folder. Keep the layout simple.

The top-level structure used by the folder named `hw01/`:

- One folder named `hw01`
  - with `README.md`, `shell/`, `screen/`, and `code/`
- A complete `README.md`
  - with the required sections, HTML required files, Redis usage, kiosk sim design, idempotency strategy, and AI use
- Implementation in `code/`
  - only relevant files needed to run the project
- Evidence in `shell/`
  - only relevant screenshots
- Evidence in `screen/`
  - only relevant short videos
- Reasonable `.gitignore` rules
  - so that `__pycache__`, `.DS_Store`, `.venv`, `node_modules`, and other local junk do not get committed

AI Use Expectation

AI can be useful and is not forbidden in this assignment.

This is not because the assignment doesn't matter. It is because the assignment asks you to connect several tools and ideas quickly: Redis, queues, Docker, Flask, HTTP, HTML, kiosk-style interfaces, a small worker loop, and idempotency. Using AI to scaffold, debug, explain errors, draft tests, or iterate on UI can be a reasonable way to learn.

You are still responsible for:

- understanding the code you submit
- verifying that the system actually works
- checking that duplicate requests do not create duplicate completed work
- accurately describing how you used AI

Good ways to use AI in this assignment:

- ask for a starter Dockerfile
- ask for help writing an HTML form submission flow
- ask for help debugging Redis connection errors
- ask for a simple worker loop
- ask for a simple kiosk simulator loop that sends 50 clients in parallel
- ask how to explain idempotency in your own words

Bad use of AI:

- pasting in code you do not understand
- trusting generated code without testing it
- claiming behavior works when you did not actually verify it

Grading

Teaching staff should primarily grade the submission by reading `README.md`.

The goal is fast, high-signal grading based on the document and the linked evidence.

`hw01/README.md` is the main artifact. We may also open `shell/` and `screen/` if the written evidence is not enough to understand what you built and why.

`README.md` should be readable in a minute or two if expected during normal grading. If simple architecture diagrams are available, staff may use them but keep it lightweight. Short bullet lists are usually better than long prose.

`shell/` and `screen/` are optional. Use them to make the system easier to understand and easier to verify.

Screenshots should show important moments such as:

- the kiosk page
- the kiosk simulator logs
- the monitoring dashboard
- successful order completion
- repeated kiosk activity
- a duplicate submission being handled safely
- worker logs or status output

Short videos are useful when they show a flow that is hard to explain in one screenshot. Keep them short and focused.

Suggested video ideas:

- placing an order from the kiosk page
- watching the kiosk simulator generate concurrent requests
- watching the monitoring dashboard update
- showing duplicate protection working

`code/` is for the actual implementation. Keep it runnable and reasonably organized.

Typical contents may include:

- `compose.yml`
- `.env.example`
- `api/`
- `worker/`
- `kiosk-sim/`
- `shared/`

Generated files, local caches, and unrelated scratch files should not be included.

Teaching Staff Guidance

Knock-Out Checklist

Check these first:

- `README.md` exists at the root
- `README.md` is present
- `README.md` includes student name, GitHub email, and SPIRE ID
- `README.md` includes enough evidence to review
- `shell/`, `screen/`, and `code/` exist
- `README.md` includes an AI Use Statement

If basic submission elements are missing, note that before doing detailed grading.

Quick Grading Rubric

Criterion	Exceeds	Meets	Needs Improvement	Below Expectations	No Submission
Submission completeness	`README.md` is complete, well-organized, and makes the submitted evidence easy to review quickly	`README.md` is present and covers the required sections with enough to review	`README.md` is present but missing important sections or makes review harder than it should be	`README.md` is present but unclear, thin, or missing some important parts	No meaningful `README.md` submission
Architecture explanation	The document explains the system clearly, with strong reasoning about queues, state, and technical choices	The document explains the system well enough to understand the main architecture and design choices	The explanation is partly hard to follow or leaves important gaps	The explanation is weak, vague, or hard to follow	No meaningful explanation of the system
Evidence of working behavior	Linked screenshots and videos make the main flows easy to see and easy to trust	Linked screenshots and videos show the main flows but require some interpretation	Some evidence is linked, but it is incomplete, weak, or hard to interpret	Evidence is sparse, confusing, or does not clearly support the claimed behavior	No meaningful evidence was provided
Correctness reasoning	The document gives a clear, convincing explanation of why duplicate work is avoided and idempotency	The document explains the main correctness ideas well enough to support the submission	Correctness reasoning is partly present, but important details are missing	Correctness reasoning is weak, mostrated, or mostly missing	No meaningful reasoning about correctness.
Professional communication	The document is easy to review quickly, with clear headings, tables, or bullets, and evidence linked where it matters	The document is readable and reasonably organized, but could be tighter or harder to review quickly	Communication is uneven or hard to scan in places, with some avoidable ambiguity	Communication is poorly organized or hard to find the important information	No meaningful communication artifact

Fast Grading Guidance

- Start with `README.md`, not everything else.
- Read `README.md` first.
- Only open `shell/`, `screen/`, and `code/` when the written evidence is not enough.
- Use `README.md` to judge scope and clarity.
- Use `shell/` and `screen/` to check whether the claimed behavior actually happens.
- Prioritize whether the document makes the system easy to understand and easy to verify.

Choose the category that matches what you saw in the submission quickly. Do not worry about mistakes. If a student seems a mistake or has a valid argument for a different grade assignment the submitted evidence, they will contact you.

Copyright © 2026 Tim Richards. Built with Decisecore.